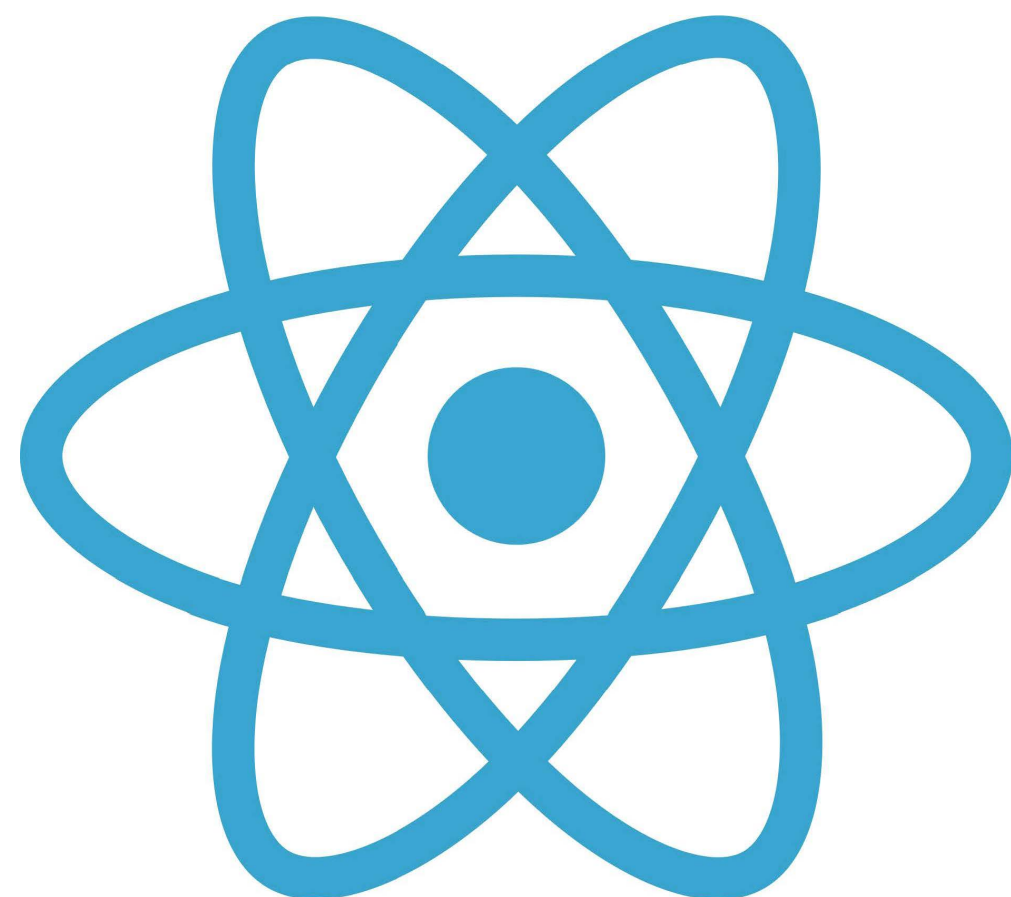




React

REACTJS

ReactJs es una librería de JavaScript para desarrollar interfaces de usuario.

El elemento más importante de React es el **componente**, que es, en esencia, una pieza de la interfaz de usuario. Como norma general, al diseñar una aplicación con React, lo que estamos haciendo es crear componentes independientes y reutilizables para, poco a poco, crear interfaces de usuario más complejas.

El Secreto de React JS

La velocidad y gran performance con la que cuenta esta librería es que trabaja con un Virtual DOM (Document Object Model), un DOM es el entorno en donde se imprimen los elementos HTML para que los usuarios puedan verlo en el navegador, normalmente se cargan estos elementos directamente en el DOM del navegador sin nada virtual, React JS utiliza su propio Virtual DOM.

REACTJS

React JS: Instalación

Podemos montar una aplicación de forma artesanal instalando dependencias por dependencia o utilizar un **boiler plate** ejecutando en la consola de comando el comando:

npm create-react-app nombre-aplicacion

* También se puede instalar React mediante otras herramientas de tooling como **Vite**

REACTJS / COMPONENTES

- * Componentes Funcionales
- * Componentes de Clase

```
1  export class App extends React.Component {  
2    constructor(props) {  
3      super(props);  
4    }  
5  
6    render() {  
7      const {  
8        props,  
9      } = this;  
10  
11      return (  
12        <div className="App">  
13          <h1>Hola mundo</h1>  
14          <h2>Soy { props.nombre}</h2>  
15        </div>  
16      );  
17    }  
18  }
```

* Componente de clase :

Es una clase de javascript que extiende la clase Component de React. retornan un JSX

REACTJS / COMPONENTES



```
1  export function App(props) {  
2    return (  
3      <div className="App">  
4        <h1>Hola mundo</h1>  
5        <h2>Soy { props.nombre}</h2>  
6      </div>  
7    );  
8  }
```

* Componente Funcional

Función de Javascript/ES6 que retorna un elemento de React (JSX / Javascript Xml)

REACTJS / COMPONENTES

Características de componentes:

- tiene que retornar un elemento JSX
- Debe comenzar con una letra Mayúscula
- Puede recibir valores si es necesarios (Los props)
- Los props se envían desde el padre al hijo

```
1  export function App(props) {  
2    return (  
3      <div className="App">  
4        <h1>Hola mundo</h1>  
5        <h2>Soy { props.nombre}</h2>  
6      </div>  
7    );  
8  }
```

REACTJS / COMPONENTES

Estado de un componente // Lo veremos en los Hooks

Representación del conjunto de propiedades de un componente y sus valores actuales en un momento específico de la aplicación que se está mostrando en la pantalla.

Estado vs **props**

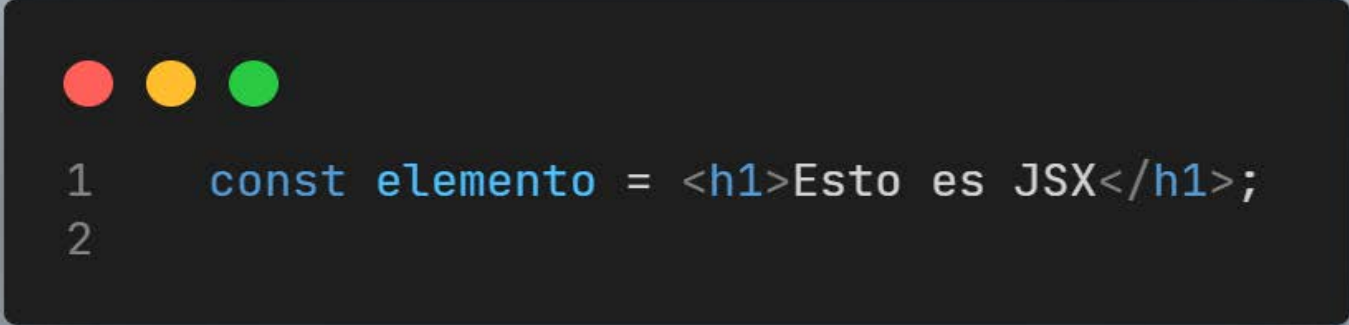
Mientras que las props son los datos que podemos pasarle a un componente o elemento React desde afuera, un estado se conforma por los datos internos que un componente puede manejar.

REACTJS / JSX

¿Qué es JSX?

JSX es una extensión de sintaxis de JavaScript que nos permite mezclar JS y HTML (XML), de ahí su nombre **JavaScript XML**. Nos permite describir en JS como se verán los componentes usando un código parecido al HTML.

Nuestro código JSX es compilado por un “**Transpiler**”, que quiere decir que toma un código de un lenguaje y lo compila al mismo código en otro lenguaje. Este “Transpiler”, llamado Babel JS, compila el código JSX a código JS.



```
1  const elemento = <h1>Esto es JSX</h1>;  
2
```


REACTJS / JSX

- Con JSX, podemos crear y usar cualquier elemento HTML.
- Hay ciertos atributos en HTML que son los mismos que palabras reservadas de JS, por eso al momento de usar estos atributos, tendríamos que cambiarlos en nuestro JSX. Ejemplo: class por **className** , for por **htmlFor**. Todas las etiquetas se deben cerrar ya sea con una etiqueta de cierre <div> </div> o por auto cerrado
- El atributo style acepta un objeto javascript con propiedades CSS con la notación del **camelCase**.
- JSX sólo permite retornar un único elemento, por lo que todos los elementos deben estar envueltos dentro de un mismo elemento.
- Podemos usar corchete **{ }** para insertar expresiones de JavaScript en nuestro código JSX.

REACTJS / JSX



```
1  function Pieza() {  
2  
3      const edad = 17;  
4      return (  
5          <>  
6              <div className="contenedor">  
7                  <h1>El valor es {6 * 3 / 2} </h1>  
8                  <h2>Soy {edad > 18 ? 'mayor' : 'menor'} de edad</h2>  
9                  <img src={miFoto} alt="" />  
10             </div>  
11         </>  
12     )  
13 }  
14
```

REACTJS / JSX

Ejemplo: uso de estilos almacenados en una variable



```
1  function App(props) {  
2    const miEstilo = {  
3      fontSize: "5rem",  
4      backgroundColor: "red"  
5    }  
6    return (  
7      <div className="App">  
8        <h1 style={ miEstilo}>Hola mundo</h1>  
9      </div>  
10   );  
11 }  
12  
13 export default App;
```

REACTJS / JSX

Ejemplo: uso de estilos en linea



```
1  function App(props) {  
2  
3    return (  
4      <div className="App">  
5        <h1 style={{ fontSize: "5rem", backgroundColor: "green" }} >Hola mundo</h1>  
6      </div>  
7    );  
8  }  
9  
10 export default App
```

*: La primera llave corresponde al código JS que le pasamos un objeto que a su vez tiene llaves uso de estilos en linea